DOCUMENT 06

NLP and AI Specification

Pipeline algorithms, scoring model, formulas, evaluation, and failure modes

Study Topic Extractor

Version: 1.0
Status: Initial Specification
Date: September 28, 2026

Turn long study material into a focused study roadmap.

Table of Contents

1. Overview and Design Principles . . . 3

1.1 Core Data Structures . . . 3

2. Text Preprocessing . . . 5

2.1 Order of Operations . . . 5

2.2 Sectioning . . . 5

3. N-grams . . . 6

3.1 Generation . . . 6

3.2 Identifying Meaningful Multi-Word Concepts . . . 6

4. TF-IDF . . . 7

4.1 Formulas . . . 7

4.2 Why TF-IDF . . . 7

5. Keyword Extraction . . . 8

6. Topic Extraction . . . 9

7. Topic Scoring . . . 10

7.1 Components . . . 10

7.2 Final Score . . . 10

7.3 Worked Example A: Full Weights (Phase 2) . . . 10

7.4 Worked Example B: MVP (Semantic Disabled) . . . 10

7.5 Illustrative Output for the Reference Sample . . . 10

8. Priority Classification . . . 12

9. Semantic Embeddings (Phase 2) . . . 13

10. Topic Clustering (Phase 2) . . . 14

11. Topic Hierarchy (Phase 2) . . . 15

11.1 Evidence Score . . . 15

11.2 Algorithm . . . 15

12. Topic Relationship Graph (Phase 2) . . . 16

13. Summary Generation . . . 17

13.1 Extractive Algorithm . . . 17

13.2 Hybrid Mode (Optional) . . . 17

14. Question Generation (Phase 2) . . . 18

15. Long-Document Processing . . . 19

16. Evaluation Methodology 20

16.1 Gold Dataset 20

16.2 Matching 20

16.3 Metrics 20

16.4 Targets and Procedure 20

17. Failure Cases and Mitigations 21

18. Reproducibility, Versioning, Acceptance Criteria 22

1. Overview and Design Principles

The NLP layer converts raw study text into ranked, explainable topics. It is **classical NLP first**: spaCy for linguistic analysis, scikit-learn for TF-IDF and clustering, and (Phase 2) Sentence Transformers for semantic signals. No LLM is required anywhere in the core pipeline.

- **Deterministic:** no randomness in scoring. Clustering uses fixed `random_state=42`. Ties are broken by occurrences then alphabetically.
- **Explainable:** each topic exposes its component scores (`tfidf`, `frequency`, `position`, `semantic`).
- **Configurable:** weights, thresholds and limits come from configuration, never from constants inside code.
- **Graceful degradation:** if a Phase 2 stage is unavailable, the pipeline continues and adds a warning.
- **Versioned:** every result carries `pipeline_version`; changing any algorithm or default bumps it.

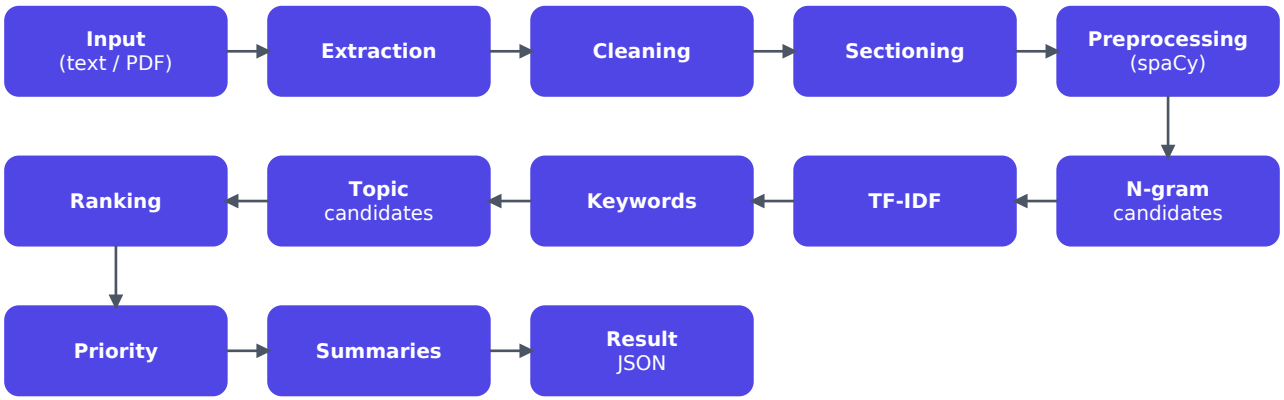


Figure 1. MVP pipeline stages (module names in the Technical Requirements)

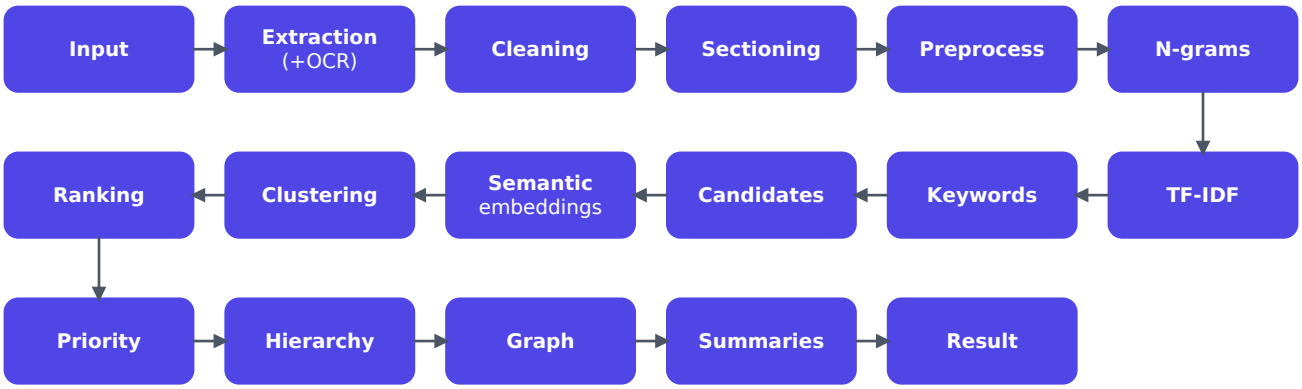


Figure 2. Full pipeline including Phase 2 stages

1.1 Core Data Structures

Object	Fields
Sentence	`text`, `start_char`, `end_char`, `section_index`
Token	`text`, `lemma`, `pos`, `is_stop`, `start_char`
Section	`index`, `heading`, `start_char`, `end_char`, `token_count`
Candidate	key (normalized lemma sequence), `surface` (most frequent original form), `ngram_size`, `count`, `first_char`, `section_ids`, `in_heading`, `in_definition`

Object	Fields
Topic	Candidate + `tfidf_c`, `frequency_c`, `position_c`, `semantic_c`, `score`, `priority`, `rank`, `summary`, `keywords`, `parent_id`, `cluster_id`

2. Text Preprocessing

2.1 Order of Operations

1. **Normalization:** Unicode NFKC; replace non-breaking spaces; normalize quotes, dashes, ellipsis; remove null and control characters (keep `\n`, `\t`).
2. **Noise removal:** strip URLs, e-mail addresses, page markers (`Page 3 of 20`), repeated header/footer lines (> 60% of pages), table-of-contents dot leaders, isolated page numbers.
3. **Whitespace handling:** collapse runs of spaces; join hyphenated line breaks (`classi-\nfication` to `classification`); convert single newlines inside paragraphs to spaces; keep blank lines as paragraph breaks.
4. **Duplicate removal:** drop exact duplicate sentences (case-insensitive) after the first occurrence for counting purposes while retaining offsets of the first.
5. **Sentence segmentation:** spaCy `sentencizer`/parser; protect abbreviations (e.g., i.e., Fig., Eq.); minimum sentence length 3 tokens.
6. **Tokenization:** spaCy tokenizer; hyphenated compounds kept as one token when both sides alphabetic (`k-means`, `decision-tree` becomes `decision tree` lemma key).
7. **Lemmatization:** spaCy lemmatizer; lowercased lemma used for keys (`trees` to `tree`); original surface form retained for display.
8. **Stopword removal:** spaCy list plus NLTK English stopwords plus academic filler list (`chapter`, `figure`, `table`, `example`, `following`, `shown`, `using`, `also`, `however`). Stopwords are dropped as single candidates and never allowed at phrase edges; internal stopwords allowed only for the pattern `noun of noun` in P2.
9. **Punctuation handling:** removed from candidates except internal hyphens, apostrophes in words, and `+ # .` inside known tokens (C++, C#, .NET, Node.js).
10. **Number handling:** pure numbers dropped; alphanumeric technical tokens kept (1NF, 2NF, 3NF, BCNF, IPv6, H2O). Years dropped.

2.2 Sectioning

Sections are detected before preprocessing. **Heading heuristics:** a line is a heading if it is <= 12 words, has no terminal period, and (is ALL CAPS, or Title Case, or starts with a numbering pattern such as `1.`, `1.2`, `Chapter 3`, `Unit II`). Sections shorter than 80 tokens merge with the previous section; sections longer than 1,200 tokens are split by the long-document procedure (Section 15). If no headings exist, the document is windowed.

3. N-grams

An **n-gram** is a contiguous sequence of n tokens. Unigrams capture single terms (`algorithm`), bigrams two-word concepts (`decision tree`), trigrams three-word concepts (`support vector machine`, `natural language processing`).

3.1 Generation

For every sentence, generate 1-, 2- and 3-grams over **lemmas**, keeping character offsets. Longer n-grams are not generated because they rarely represent study concepts and inflate the candidate space.

3.2 Identifying Meaningful Multi-Word Concepts

Filter	Rule	Rationale
POS pattern	Must match `(ADJ|NOUN|PROPN)* NOUN|PROPN`; last token is a noun	Concepts are noun phrases
Edge filter	No stopword, pronoun, verb or number at either edge	Removes `of the model`, `is a branch`
Minimum count	n >= 2 requires count >= 2, unless in heading or definition context	Removes accidental phrases
Collocation strength	NPMI(w1..wn) >= 0.10 (bigrams; for trigrams, min NPMI of adjacent pairs)	Keeps phrases whose words occur together more than chance
Subsumption	Drop a shorter candidate if >= 80% of its occurrences are inside a kept longer candidate	Suppresses `machine`, `vector` when `support vector machine` is the concept
Acronyms	Map SVM and `support vector machine` to one key when the definition pattern `X (ABBR)` is found	Merges variants
Length boost	Multiplier m(n): 1.00, 1.15, 1.25 for n = 1, 2, 3 applied to the TF-IDF signal	Multi-word concepts receive higher importance

NPMI (normalized pointwise mutual information) for a bigram (x, y): `NPMI = ln( p(x,y) / (p(x) p(y)) ) / ( -ln p(x,y) )`, range -1 to 1. For very small documents (< 1,000 tokens) the NPMI filter is skipped and the minimum count rule is relaxed for phrases matching the POS pattern in a definitional sentence.

4. TF-IDF

Term frequency (TF) measures how often a candidate occurs in the document; **inverse document frequency (IDF)** down-weights terms that are common across many sections or in general language; **TF-IDF** is their product and highlights terms that are frequent here but not everywhere.

4.1 Formulas

```
TF(t)       = 1 + ln(count(t))                      if count(t) > 0, else 0   (sublinear)
IDF_doc(t)  = ln( (1 + N) / (1 + df(t)) ) + 1       N = #sections, df = #sections containing t
IDF_bg(t)   = ln( (1 + M) / (1 + bg_df(t)) ) + 1    M = background corpus size (background_idf.json)
                                                    unseen terms receive the maximum IDF_bg
IDF(t)      = IDF_bg(t)                             if N < 5
            = 0.5 * IDF_doc(t) + 0.5 * IDF_bg(t)    if N >= 5
raw_tfidf(t)= TF(t) * IDF(t) * m(n)                 m(1)=1.00, m(2)=1.15, m(3)=1.25
tfidf_c(t)  = raw_tfidf(t) / max over candidates of raw_tfidf   (range 0..1)
```

4.2 Why TF-IDF

- Fast, deterministic, interpretable, no training or GPU needed.
- Works on a single document when combined with a background IDF table (built once from an open general-English corpus and shipped as `data/background_idf.json`, about 50,000 terms).
- Weakness: cannot understand meaning or synonyms. This is why Phase 2 adds a semantic signal.

Implementation uses scikit-learn `TfidfVectorizer` with a custom analyzer that returns precomputed candidate keys per section (so n-gram logic stays in `ngram_extractor.py`), `norm=None`, `smooth_idf=True`, `sublinear_tf=True`; blended IDF is applied afterward.

5. Keyword Extraction

Keywords are distinctive vocabulary items shown as chips; **topics** are study concepts shown as ranked cards. The two lists overlap but are produced separately.

1. Start from all candidates after n-gram filters.
2. Rank by `raw_tfidf(t)` (already length-boosted).
3. Remove candidates whose surface form is a single generic word in the academic filler list.
4. Keep the top 30 keywords; report `count` and normalized score `raw/max`.
5. **Per-topic keywords:** for each topic, collect candidates co-occurring in the same sentences as the topic; rank by co-occurrence count times TF-IDF; keep top 3 to 5 excluding the topic itself and its sub-phrases.

Example. Input: `Supervised learning uses labeled datasets to train machine learning models.`
Keywords: Supervised Learning, Labeled Dataset, Machine Learning, Training, Models.

6. Topic Extraction

Topic candidates are drawn from four sources and merged by normalized lemma key.

Source	Example	Boost
Filtered n-gram candidates	decision tree	-
Headings and section titles	Normalization	`in_heading=true`
Subjects of definitional sentences (`X is a/an/the`, `X refers to`, `X is defined as`, `X means`)	Supervised learning	`in_definition=true`
Enumeration heads and items (`include A, B and C`, `types of X`)	algorithms; decision trees	-

Merge and cleanup: merge singular/plural and hyphenation variants; map acronyms; drop candidates that are only stopwords or generic words; cap at 60 candidates by preliminary score `0.5 tfidf_c + 0.3 freq_c + 0.2 pos_c` (semantic excluded) before final ranking. Final output returns up to `MAX_TOPICS` (default 25) with score >= 20.

Example. Input: `Classification predicts discrete categories, while regression predicts continuous values.` Topics: Classification, Regression, Discrete Categories, Continuous Values.

7. Topic Scoring

7.1 Components

Component	Definition (all in 0..1)	Notes
`tfidf_c`	As defined in Section 4	Distinctiveness
`frequency_c`	`ln(1 + count) / ln(1 + max_count)` over candidates; `count` = offset-based total occurrences of the lemma key in the document	Log scaling prevents one very common term from flattening others
`position_c`	`0.5 * earliness + 0.3 * heading_flag + 0.2 * definition_flag`; `earliness = 1 - min(1, first_char / min(len, 20000))`	Early mention, headings and definitions signal importance
`semantic_c` (P2)	`max(0, cos(e_topic, e_doc)) / max over candidates`	See Section 9

7.2 Final Score

```
Score(t) = 100 * ( w_tfidf*tfidf_c + w_freq*frequency_c + w_pos*position_c
                   + w_sem*semantic_c ) / ( sum of weights of ENABLED signals )
Rounded to 1 decimal. Range 0..100. Components are max-scaled (not min-max) so the
top topic is not forced to 100 and the lowest is not forced to 0.
Tie-break: higher occurrences, then alphabetical by normalized key.
```

Signal	Configured weight	Effective weight, semantic disabled (MVP)	Meaning
TF-IDF	0.40	0.47	Distinctiveness of the term for this document
Frequency	0.25	0.29	How often the concept occurs (log-scaled)
Position	0.20	0.24	Earliness, heading membership, definition context
Semantic relevance	0.15	0 (disabled)	Embedding similarity to the document centroid (Phase 2)

The effective MVP weights are `w_i / 0.85` = 0.4706, 0.2941, 0.2353 (shown rounded). The pipeline always renormalizes over enabled signals; therefore enabling semantic analysis later changes scores without any code change.

7.3 Worked Example A: Full Weights (Phase 2)

Topic `Supervised Learning`: TF-IDF 0.82, Frequency 0.90, Position 0.75, Semantic 0.88.

```
(0.82 x 0.40) + (0.90 x 0.25) + (0.75 x 0.20) + (0.88 x 0.15)
= 0.328 + 0.225 + 0.150 + 0.132 = 0.835   -> Score 83.5 -> HIGH
```

7.4 Worked Example B: MVP (Semantic Disabled)

```
(0.82 x 0.40 + 0.90 x 0.25 + 0.75 x 0.20) / 0.85
= (0.328 + 0.225 + 0.150) / 0.85 = 0.703 / 0.85 = 0.8271  -> Score 82.7 -> HIGH
```

7.5 Illustrative Output for the Reference Sample

Sample input (about 310 characters, one section):

```
Machine learning is a branch of artificial intelligence.
Supervised learning uses labeled datasets to train models.
Classification predicts discrete categories, while regression predicts continuous values.
Common algorithms include decision trees, support vector machines, and linear regression.
```

Rank	Topic	Illustrative score	Priority
1	Machine Learning	95	HIGH
2	Supervised Learning	91	HIGH
3	Classification	87	HIGH
4	Regression	84	HIGH
5	Decision Trees	73	MEDIUM
6	Support Vector Machines	70	MEDIUM
7	Linear Regression	64	MEDIUM

TESTING NOTE. These scores illustrate the intended behavior and ordering. Automated tests must assert (a) the top-7 set, (b) that Machine Learning and Supervised Learning are HIGH, and (c) monotonic ordering constraints, not the exact numbers, because scores depend on the shipped background IDF table.

8. Priority Classification

Priority	Score range	Meaning	UI
HIGH	80 to 100	Study first. Central concepts of the document.	Red, filled-circle icon, label HIGH
MEDIUM	50 to 79	Review after high-priority topics.	Amber, half-circle icon, label MEDIUM
LOW	0 to 49	Supporting or peripheral concepts.	Cyan, ring icon, label LOW

- Thresholds are configuration (`PRIORITY_HIGH_MIN=80`, `PRIORITY_MEDIUM_MIN=50`), boundaries inclusive at the lower edge (80 is HIGH, 50 is MEDIUM). Validation: `high > medium > 0`.
- **Absolute mode (default):** thresholds apply to Score directly, so different documents are comparable.
- **Percentile mode (optional):** top 20% HIGH, next 40% MEDIUM, rest LOW, for documents where scores are flat.
- No forced promotion: a document may legitimately have zero HIGH topics; the UI then highlights the top 3 topics as `Top picks` without changing their priority.

9. Semantic Embeddings (Phase 2)

Model: `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions, about 90 MB, CPU friendly). Pipeline: text to tokens to transformer to pooled **embedding vector**; similarity is measured with cosine similarity.

```
cos(a, b) = (a . b) / ( ||a|| * ||b|| )        range -1..1 (higher = more similar)
With L2-normalized vectors, cos(a, b) = a . b
```

- Embed each topic candidate string and each section (truncate to 256 word pieces; longer sections are embedded by sentence groups and averaged).
- **Document centroid:** `e_doc = normalize(mean(section embeddings))`.
- **Semantic relevance:** `semantic_c(t) = max(0, cos(e_t, e_doc)) / max_t(...)`.
- **Relatedness example:** `Predicting categories` versus `Classification` yields high cosine although the words differ.
- Batch encode (batch size 32), `torch.set_num_threads(2)`, model loaded once at startup, embeddings cached per request.
- **Fallback:** if the model is unavailable, skip the stage, set `semantic_enabled=false`, renormalize weights, and add warning `SEMANTIC_DISABLED`.

10. Topic Clustering (Phase 2)

Algorithm	Strengths	Weaknesses	Decision
K-Means	Fast, simple	Needs k in advance; assumes spherical clusters; cosine not native	Fallback only, with k = clamp(round(sqrt(n/2)), 2, 8) on normalized vectors
Hierarchical (agglomerative)	No k needed with a distance threshold; deterministic; supports cosine; pairs naturally with hierarchy	O(n^2), fine for n <= 60	**Default:** average linkage, cosine distance, threshold 0.45
DBSCAN	Finds arbitrary shapes; labels noise	Sensitive to eps and min_samples; unstable on tiny sets	Not used initially; evaluate later

Clusters do not alter scores. They are labeled by their highest-scoring member and used for grouping in the UI and as evidence for hierarchy. Clustering is skipped for fewer than 6 topics. Singletons are allowed.

11. Topic Hierarchy (Phase 2)

Goal: assign each topic at most one parent so the result forms a forest with maximum depth 4.

11.1 Evidence Score

```
e(p, c) = 0.40 * lex(p, c) + 0.30 * pat(p, c) + 0.30 * max(0, cos(e_p, e_c))
lex = 1.0 if tokens(p) is a proper subset of tokens(c)    (regression, linear regression)
      0.5 if head(p) == head(c) and generality(p) > generality(c)
      0.0 otherwise
pat = 1.0 if a sentence matches "p (includes|such as|consists of|types of|kinds of|
      comprises) ... c" or an enumeration under p, else 0.0
generality(t) = fraction of sections containing t   (parent needs generality >= child)
```

11.2 Algorithm

1. Consider topic pairs among the top 40 topics; keep pairs with `generality(p) >= generality(c)`, `p != c`.
2. For each topic c, choose the parent p with maximum `e(p, c)`; require `e >= 0.45` (configurable). Tie: higher score.
3. Build a NetworkX DiGraph; detect cycles with `find_cycle`; remove the lowest-evidence edge in each cycle until acyclic.
4. Clamp depth to 4 by re-attaching deeper nodes to their grandparent.
5. Topics without a parent are roots. Store `parent_id`, `children_ids`, `depth`.

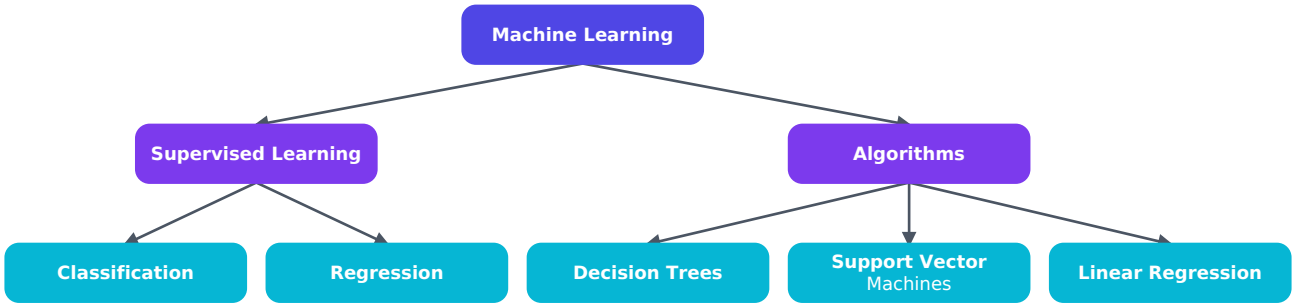


Figure 3. Expected hierarchy for the reference sample

In the reference sample, Supervised Learning attaches to Machine Learning (lexical head and semantic evidence), and the enumeration `Common algorithms include decision trees, support vector machines, and linear regression` gives Algorithms as parent of the three algorithm topics. Regression as parent of Linear Regression is also supported lexically; when both Algorithms and Regression are candidates, the higher evidence wins (pattern evidence beats lexical containment), and the alternative is exposed as a `semantic` graph edge.

12. Topic Relationship Graph (Phase 2)

Element	Definition
Node	`{id, label, score, priority, cluster_id, depth}`; radius scales with score; color by priority (with shape cue)
Edge: hierarchy	Directed parent to child; weight = evidence `e(p, c)`
Edge: semantic	Undirected; cosine >= 0.55; keep top 3 per node; weight = cosine
Edge: cooccurrence (optional)	Two topics in the same sentence >= 2 times; weight = normalized count
Limits	Max 40 nodes, max 120 edges; lower-score nodes removed first
Construction	NetworkX builds and prunes; API returns JSON `{nodes, edges}`; layout is computed client side by Cytoscape (`cose`)

13. Summary Generation

Approach	Pros	Cons	Use
Extractive	Deterministic, free, no hallucination, fast	Less fluent	**Default and always-available fallback**
LLM-generated	Fluent, can rephrase	Cost, latency, hallucination risk, privacy, non-deterministic	Never used alone
Hybrid	Fluent but grounded	Needs LLM	Optional (Phase 2, `ENABLE_LLM=true`)

13.1 Extractive Algorithm

```
for each topic: candidate sentences = sentences containing the topic (lemma match)
sentence_score = 3.0 * is_definition(sentence, topic)      # X is a / refers to / is defined as
               + 1.0 * topic_in_first_5_tokens
               + 0.5 * min(1, other_high_topics_in_sentence / 2)
               + 0.5 * (8 <= token_count <= 40)
               - 1.0 * (token_count > 60)
choose top sentence (tie: earlier); if < 120 chars and next sentence adds the topic
context, append it; truncate at sentence boundary to <= 280 chars.
```

13.2 Hybrid Mode (Optional)

- Input to the LLM: only the selected source sentences (max 3) and the topic name, wrapped as data, never as instructions.
- System prompt fixes the task: `Rewrite the sentences as a clear 1-2 sentence explanation for a student. Use only the provided text. If insufficient, reply INSUFFICIENT.`
- Output validation: length <= 280 chars; at least 60% of content lemmas must appear in the source sentences; otherwise discard and use the extractive summary.
- Timeout 8 s, one retry; any failure falls back to extractive. See Security Specification for prompt-injection controls.

14. Question Generation (Phase 2)

Questions are generated per HIGH topic (and MEDIUM on request), 3 to 6 each, from templates selected by the topic's role in the hierarchy, with optional LLM paraphrasing.

Type	Template	Condition
definition	What is {topic}?	Topic has a definition sentence
explanation	Explain {topic} with an example.	Always
list	List the types of {parent} and describe each.	Topic is a parent with >= 2 children
comparison	Differentiate between {topic} and {sibling}.	Topic has a sibling in the hierarchy
why_how	Why is {topic} important in {root}? / How does {topic} work?	Topic has depth >= 1 or verb-pattern context
application	Where is {topic} used?	Context sentence mentions use/apply/used in

Example. Topic: Normalization. Questions: What is normalization? Explain 1NF, 2NF and 3NF. What is a functional dependency? Explain lossless decomposition. Difficulty: easy for definition, medium for explanation/list, hard for comparison/why-how. Deduplicate by lemma-set similarity >= 0.8; cap per topic.

15. Long-Document Processing

1. **Chunking:** split by detected sections; any section over 1,200 tokens is split on sentence boundaries into windows of about 800 tokens with 100-token overlap.
2. **Per-chunk work:** preprocessing and candidate extraction run per chunk in a loop (bounded memory); embeddings batched.
3. **Aggregation:** occurrence counts and first positions are computed on **global character offsets** so overlapped regions are never double-counted; `section_ids` unions; `in_heading` and `in_definition` are OR-ed.
4. **Deduplication:** merge candidates with equal normalized key, acronym mappings, or (P2) cosine >= 0.92; keep the most frequent surface form.
5. **TF-IDF:** df computed across chunks/sections (Section 4).
6. **Limits:** 150,000 characters; above 30,000 characters run as an asynchronous job (P2). Complexity is linear in document length except clustering (quadratic in <= 60 topics).

16. Evaluation Methodology

16.1 Gold Dataset

Create `backend/data/gold/` with at least 30 documents (10 computer science, 5 physics/chemistry, 5 biology, 5 humanities/social science, 5 mixed or noisy PDFs). For each, one author lists 8 to 20 expected topics; a second reviewer resolves disagreements. Store as JSON: `{doc_id, source, text_path, gold_topics: [{name, priority?}]}`. Split 20 dev / 10 test; the test split is never used for tuning.

16.2 Matching

A predicted topic matches a gold topic if their normalized lemma keys are equal, or token-set Jaccard >= 0.6, or (for multi-word gold topics) the predicted key contains the gold key with >= 0.75 token overlap. Each gold topic can be matched once.

16.3 Metrics

```
Precision@K  = |TopK and Gold| / K                (how many extracted topics are relevant)
Recall@K     = |TopK and Gold| / |Gold|           (how many important topics were found)
F1@K         = 2 * P@K * R@K / (P@K + R@K)       (balance)
TopK-Acc     = |TopK and Gold| / min(|Gold|, K)    (expected topics found within top K)
HIGH-Precision = fraction of HIGH topics that match gold topics
Report macro-averages over documents for K in {5, 10, 15}.
```

Example. Gold: Machine Learning, Classification, Regression. Output: Machine Learning, Classification, Regression, Dataset, Prediction. P@5 = 3/5 = 0.60, R@5 = 3/3 = 1.00, F1@5 = 0.75.

16.4 Targets and Procedure

Metric	MVP target	Phase 2 target
F1@10	>= 0.55	>= 0.65
Top-10 accuracy	>= 0.60	>= 0.70
HIGH-Precision	>= 0.60	>= 0.70
Determinism (same output on 2 runs)	100%	100%

1. Run `python scripts/evaluate.py --split dev` after every scoring change and store the report with the `pipeline_version`.
2. **Ablations:** TF-IDF only; + frequency; + position; + semantic; with and without NPMI filter and subsumption.
3. **Weight tuning:** grid search on dev split, step 0.05, weights sum to 1; adopt new defaults only if dev F1@10 improves by >= 0.02 and test F1@10 does not regress.
4. Track error categories (missed, generic, fragment, duplicate) for qualitative review.

17. Failure Cases and Mitigations

Failure	Symptom	Mitigation
Poorly written notes	Fragments become topics	Noun-phrase and edge filters; warnings; fewer topics returned
Extremely short documents	Unstable statistics	Background IDF; SHORT_DOCUMENT warning; relaxed min count
Ambiguous concepts	One term with two meanings merged	Surface-form display; future sense disambiguation
Domain jargon	Rare terms over-ranked	Frequency and position signals dampen; background IDF cap
Scanned PDFs without OCR	No text	NO_READABLE_TEXT; OCR in P2
Repeated words / boilerplate	Inflated frequency	Header/footer removal; duplicate sentence collapse; academic filler list
Deep context needed	Misses implicit topics	Documented limitation; optional LLM assistance later
Math notation	Symbols and fragments	Strip symbols; MATH_CONTENT warning
Images and diagrams	Content invisible	Documented limitation
Long unstructured documents	Diluted topics	Windowing; section-based df
Non-English text	Poor lemmas/stopwords	Language check warning; multilingual models later
Synonyms	Duplicate concepts	P2 embedding-based deduplication

18. Reproducibility, Versioning, Acceptance Criteria

- `pipeline_version` follows semantic versioning; results include the config snapshot and model identifiers.
- Pin versions of spaCy, `en_core_web_sm`, scikit-learn, sentence-transformers, and the background IDF table hash in `requirements.txt` and build logs.

1. Same input, config and version yields byte-identical topic lists in two runs.
2. Reference sample produces the required top-7 set with Machine Learning and Supervised Learning HIGH.
3. Weights renormalize correctly when a signal is disabled (unit tested with the worked examples: 83.5 and 82.7).
4. Priority boundary tests at 49.9, 50.0, 79.9, 80.0 pass.
5. Every summary is a source substring in extractive mode and <= 280 characters.
6. Gold-set evaluation meets MVP targets on the test split before MVP release.